Phase 1 Master Document Template

Memory Scaffold + Security + Verification

📋 Phase 1 Overview

Duration: ~8 hours
Prompts: 7 (PROMPT 01-07)
Status: ✅ COMPLETED
Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems

Final Metrics

Metric	Value
Total Tests	472
Code Coverage	90.05%
Exception Classes	14
Security Policies	7
Core Interfaces	8
Documentation Files	8

Phase 1 Prompts

1. **PROMPT 01:** Development Environment Setup
2. **PROMPT 02:** TypeScript Project Initialization
3. **PROMPT 02B:** Quality Gates Setup
4. **PROMPT 03:** Exception Hierarchy + Guardrails Foundation
5. **PROMPT 04:** Memory Scaffold Implementation
6. **PROMPT 05:** Security Hardening
7. **PROMPT 06:** Tests + Coverage Enhancement
8. **PROMPT 07:** Documentation + Git + Phase 1 Completion

Key Accomplishments

- ✅ Comprehensive exception hierarchy with 14 specialized error classes
- ✅ Production-grade security with symlink detection and audit logging
- ✅ Robust memory scaffold with path validation and file operations
- ✅ 90%+ code coverage with 472 comprehensive tests
- ✅ Quality gates with Husky pre-commit hooks

- ✅ Complete documentation and architecture guides

PROMPT 01: Development Environment Setup

Phase: 1 of 4
Prompt: 01 of 07
Critical Level: 🟢 LOW
Duration Estimate: 30-60 minutes
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** N/A (First prompt)
- **Next:** PROMPT 02: TypeScript Project Initialization
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. ➡️ PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] None (this is the first prompt)
- [ ] Clean working directory
- [ ] System requirements met (Node.js compatible OS)

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Node.js installation and version verification
- pnpm package manager setup
- TypeScript global/local installation
- Development tools configuration
- Environment validation

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] Node.js v18+ installed
- [ ] pnpm installed and configured
- [ ] TypeScript installed
- [ ] Development environment verified
- [ ] Environment documentation

Success Criteria

- [ ] `node --version` returns v18 or higher
- [ ] `pnpm --version` executes successfully
- [ ] `tsc --version` returns TypeScript version
- [ ] All tools accessible from command line
- [ ] Environment ready for TypeScript development

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Files Created	[PLACEHOLDER: #]
Commands Executed	[PLACEHOLDER: #]
Tools Installed	[PLACEHOLDER: #]
Tests Run	N/A
Coverage	N/A

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:
- Environment setup notes
- Version verification logs
- Configuration files (if any)

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Next Prompt

➡️ **PROMPT 02: TypeScript Project Initialization**

PROMPT 02: TypeScript Project Initialization

Phase: 1 of 4
Prompt: 02 of 07
Critical Level: 🟡 MEDIUM
Duration Estimate: 1-2 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 01: Development Environment Setup
- **Next:** PROMPT 02B: Quality Gates Setup

- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. ➡️ PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] Node.js v18+ installed
- [ ] pnpm installed
- [ ] TypeScript available
- [ ] Project directory created

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- TypeScript configuration (tsconfig.json)
- Package.json setup with scripts
- Project directory structure
- ESLint and Prettier configuration
- Initial testing framework setup (Vitest)
- Build and development scripts

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] tsconfig.json with strict settings
- [ ] package.json with dependencies and scripts
- [ ] Directory structure (src/, tests/, dist/)
- [ ] ESLint configuration
- [ ] Prettier configuration

- [ ] Basic build pipeline
- [ ] Testing framework configured

Success Criteria

- [ ] `pnpm install` completes without errors
- [ ] `pnpm build` successfully compiles TypeScript
- [ ] `pnpm test` runs test framework
- [ ] `pnpm lint` executes ESLint
- [ ] Project structure follows best practices
- [ ] TypeScript strict mode enabled

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Configuration Files Created	[PLACEHOLDER: #]
Dependencies Installed	[PLACEHOLDER: #]
Scripts Added	[PLACEHOLDER: #]
Directory Structure Depth	[PLACEHOLDER: #]
Initial Tests	[PLACEHOLDER: #]
Coverage	[PLACEHOLDER: X%]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:

- `package.json`

- `tsconfig.json`
- `.eslintrc.json` or `.eslintrc.js`
- `.prettierrc`
- `vitest.config.ts`
- Directory structure: `src/`, `tests/`, `dist/`

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Next Prompt

➡️ **PROMPT 02B: Quality Gates Setup**

PROMPT 02B: Quality Gates Setup

Phase: 1 of 4
Prompt: 02B of 07
Critical Level: 🟡 MEDIUM
Duration Estimate: 30-60 minutes
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 02: TypeScript Project Initialization
- **Next:** PROMPT 03: Exception Hierarchy + Guardrails Foundation
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. ➡️ PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] PROMPT 02 completed
- [ ] Git initialized (or will be initialized)
- [ ] Package.json with scripts configured
- [ ] ESLint and Prettier configured

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Husky installation and configuration
- Git hooks setup (pre-commit)
- lint-staged configuration
- check:all script implementation
- Quality gates enforcement
- Pre-commit validation workflow

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] Husky installed and initialized
- [ ] Pre-commit hook configured
- [ ] lint-staged configuration file
- [ ] check:all script in package.json
- [ ] Git hooks working and enforced
- [ ] Quality gates documentation

Success Criteria

- [ ] Husky hooks execute on commit
- [ ] lint-staged runs on staged files
- [ ] check:all runs lint, type-check, and tests
- [ ] Invalid commits are blocked
- [ ] Quality gates prevent bad code from being committed
- [ ] Team workflow documented

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Hooks Configured	[PLACEHOLDER: #]
Quality Gates Added	[PLACEHOLDER: #]
Scripts Modified	[PLACEHOLDER: #]
Test Commits	[PLACEHOLDER: #]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:
- `.husky/pre-commit`
- `.lintstagedrc.json` or `.lintstagedrc.js`
- Updated `package.json` with check:all script
- `.husky/_/` directory with Husky internals

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Next Prompt

➡️ **PROMPT 03: Exception Hierarchy + Guardrails Foundation**

PROMPT 03: Exception Hierarchy + Guardrails Foundation

Phase: 1 of 4
Prompt: 03 of 07
Critical Level: 🔴 HIGH
Duration Estimate: 2-3 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 02B: Quality Gates Setup
- **Next:** PROMPT 04: Memory Scaffold Implementation
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. ➡️ PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] PROMPT 02 completed
- [ ] PROMPT 02B completed
- [ ] TypeScript project fully initialized
- [ ] Testing framework operational
- [ ] Quality gates active

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Base exception class hierarchy
- 14 specialized exception classes
- 7 security policies (Path, Injection, Symlink, Lock, Audit, Encoding, Size)
- 8 core interfaces (Policy, PolicyResult, AuditEntry, ValidationError, etc.)
- Comprehensive error handling strategy
- Production-grade exception system

Expected Exception Classes:
1. MemoryScaffoldError (base)
2. ValidationError
3. PathValidationError
4. FileOperationError
5. DirectoryOperationError
6. SecurityPolicyError
7. SymlinkError
8. InjectionError
9. FileNotFoundError
10. PermissionError
11. LockError
12. AuditError
13. EncodingError
14. ConfigurationError

Expected Policies:
1. PathSecurityPolicy
2. InjectionPreventionPolicy
3. SymlinkPolicy
4. FileLockingPolicy
5. AuditLoggingPolicy
6. EncodingValidationPolicy
7. SizeLimitPolicy

Expected Interfaces:
1. IPolicy
2. IPolicyResult
3. IAuditEntry
4. IValidationError
5. IPathValidationResult
6. IFileOperationResult
7. ISecurityContext
8. IPolicyViolation

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] Complete exception hierarchy
- [ ] All 14 exception classes implemented
- [ ] All 7 security policies implemented
- [ ] All 8 interfaces defined

- [ ] Comprehensive test suite (201+ tests)
- [ ] 96%+ code coverage
- [ ] Exception documentation
- [ ] Policy usage examples

Success Criteria

- [ ] All exception classes properly extend base class
- [ ] Exception serialization/deserialization works
- [ ] All policies implement IPolicy interface
- [ ] Policies can be enforced individually or combined
- [ ] Tests cover all exception types
- [ ] Tests cover all policy scenarios
- [ ] Code coverage ≥ 96%
- [ ] No circular dependencies
- [ ] Type safety maintained

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Exception Classes Created	14 (target) / [PLACEHOLDER: actual]
Security Policies Implemented	7 (target) / [PLACEHOLDER: actual]
Interfaces Defined	8 (target) / [PLACEHOLDER: actual]
Tests Written	201+ (target) / [PLACEHOLDER: actual]
Code Coverage	96%+ (target) / [PLACEHOLDER: actual %]
Source Files Created	[PLACEHOLDER: #]
Test Files Created	[PLACEHOLDER: #]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:

- `src/exceptions/base.exception.ts`
- `src/exceptions/validation.exception.ts`
- `src/exceptions/path-validation.exception.ts`
- `src/exceptions/file-operation.exception.ts`
- `src/exceptions/directory-operation.exception.ts`
- `src/exceptions/security-policy.exception.ts`
- `src/exceptions/symlink.exception.ts`
- `src/exceptions/injection.exception.ts`
- `src/exceptions/file-not-found.exception.ts`
- `src/exceptions/permission.exception.ts`
- `src/exceptions/lock.exception.ts`
- `src/exceptions/audit.exception.ts`
- `src/exceptions/encoding.exception.ts`
- `src/exceptions/configuration.exception.ts`
- `src/policies/*.policy.ts` (7 files)
- `src/interfaces/*.interface.ts` (8 files)
- `tests/exceptions/*.test.ts`
- `tests/policies/*.test.ts`

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Architecture Decisions

[PLACEHOLDER: User will document key architectural decisions made]

Expected Decisions:
- Exception hierarchy structure
- Policy enforcement strategy
- Interface design patterns
- Error code conventions
- Serialization format

Next Prompt

➡️ **PROMPT 04: Memory Scaffold Implementation**

PROMPT 04: Memory Scaffold Implementation

Phase: 1 of 4
Prompt: 04 of 07
Critical Level: 🔴 HIGH
Duration Estimate: 2-3 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 03: Exception Hierarchy + Guardrails Foundation
- **Next:** PROMPT 05: Security Hardening
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. ➡️ PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed

- [ ] PROMPT 02 completed
- [ ] PROMPT 02B completed
- [ ] PROMPT 03 completed
- [ ] Exception hierarchy fully implemented
- [ ] All policies and interfaces available
- [ ] Tests passing for exceptions and policies

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- PathValidator implementation
- FileOperations implementation
- DirectoryOperations implementation
- MemoryScaffold main class implementation
- Integration of all policies
- Comprehensive operation coverage (read, write, delete, move, copy, etc.)
- Policy enforcement in operations
- Error handling and recovery

Expected Core Components:
1. **PathValidator:**
- Path sanitization
- Traversal attack prevention
- Allowed paths validation
- Normalization
- Security checks

1. **FileOperations:**
   - Read file (text/binary)
   - Write file (text/binary)
   - Append file
   - Delete file
   - Move file
   - Copy file
   - File existence checks
   - File metadata operations

2. **DirectoryOperations:**
   - Create directory
   - Delete directory (recursive/non-recursive)
   - List directory contents
   - Directory existence checks
   - Directory traversal
   - Directory metadata operations

3. **MemoryScaffold:**
   - Unified API for all operations
   - Policy orchestration
   - Configuration management
   - Operation logging
   - Transaction support (if applicable)

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] PathValidator fully implemented
- [ ] FileOperations fully implemented
- [ ] DirectoryOperations fully implemented
- [ ] MemoryScaffold main class implemented
- [ ] All operations integrated with policies
- [ ] Comprehensive test suite (299+ tests total)
- [ ] 88%+ code coverage
- [ ] Integration tests
- [ ] API documentation
- [ ] Usage examples

Success Criteria

- [ ] All path validation works correctly
- [ ] All file operations work correctly
- [ ] All directory operations work correctly
- [ ] Policies are enforced on all operations
- [ ] Exceptions are thrown appropriately
- [ ] Tests cover happy paths and error cases
- [ ] Code coverage ≥ 88%
- [ ] No security vulnerabilities
- [ ] Performance acceptable for typical operations
- [ ] Memory leaks prevented

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Core Components Implemented	4 (target) / [PLACEHOLDER: actual]
Total Operations Implemented	[PLACEHOLDER: #]
Tests Written (Cumulative)	299+ (target) / [PLACEHOLDER: actual]
Code Coverage	88%+ (target) / [PLACEHOLDER: actual %]
Source Files Created	[PLACEHOLDER: #]
Test Files Created	[PLACEHOLDER: #]
Integration Tests	[PLACEHOLDER: #]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:
- `src/core/path-validator.ts`
- `src/core/file-operations.ts`
- `src/core/directory-operations.ts`
- `src/core/memory-scaffold.ts`
- `src/config/memory-scaffold.config.ts`
- `tests/core/path-validator.test.ts`
- `tests/core/file-operations.test.ts`
- `tests/core/directory-operations.test.ts`
- `tests/core/memory-scaffold.test.ts`
- `tests/integration/memory-scaffold.integration.test.ts`

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Architecture Decisions

[PLACEHOLDER: User will document key architectural decisions made]

Expected Decisions:
- Sync vs async operations strategy
- Error handling patterns
- Policy enforcement points
- Configuration approach
- Transaction support (if any)

Performance Notes

[PLACEHOLDER: User will note any performance considerations or optimizations]

Next Prompt

➡️ **PROMPT 05: Security Hardening**

PROMPT 05: Security Hardening

Phase: 1 of 4
Prompt: 05 of 07
Critical Level: 🔴 HIGH
Duration Estimate: 1-2 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 04: Memory Scaffold Implementation
- **Next:** PROMPT 06: Tests + Coverage Enhancement
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization

3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. ➡️ PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] PROMPT 02 completed
- [ ] PROMPT 02B completed
- [ ] PROMPT 03 completed
- [ ] PROMPT 04 completed
- [ ] Memory scaffold fully implemented
- [ ] All core operations working
- [ ] Basic security policies in place

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Symlink detection and prevention
- Advanced path traversal protection
- Audit logging enhancement
- File locking mechanisms
- Race condition prevention
- Input sanitization hardening
- Security policy refinement
- Vulnerability testing

Expected Security Enhancements:
1. **Symlink Protection:**
- Detect symbolic links
- Prevent symlink attacks
- Resolve real paths
- Block operations on symlinks (configurable)

1. **Audit Logging:**
   - Comprehensive operation logging
   - Security event logging
   - Timestamp and context capture
   - Log rotation and management
   - Tamper-proof logs

2. **File Locking:**
   - Exclusive locks for write operations
   - Shared locks for read operations
   - Lock timeout handling
   - Deadlock prevention
   - Lock cleanup on errors

3. **Additional Hardening:**
   - Race condition mitigation
   - Time-of-check-time-of-use (TOCTOU) prevention
   - Input validation strengthening
   - Size limit enforcement
   - Encoding validation

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] Symlink detection fully implemented
- [ ] Enhanced audit logging system
- [ ] File locking mechanism implemented
- [ ] Race condition mitigations in place
- [ ] Security tests expanded (362+ tests total)
- [ ] 87%+ code coverage maintained
- [ ] Security audit documentation
- [ ] Threat model document
- [ ] Security best practices guide

Success Criteria

- [ ] Symlinks are detected and handled appropriately
- [ ] All operations are logged to audit trail
- [ ] File locking prevents concurrent write conflicts
- [ ] Race conditions are mitigated
- [ ] TOCTOU vulnerabilities addressed
- [ ] Security tests pass (362+ tests)
- [ ] Code coverage ≥ 87%
- [ ] No known vulnerabilities remain
- [ ] Security review completed

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Security Features Added	[PLACEHOLDER: #]
Vulnerabilities Fixed	[PLACEHOLDER: #]
Tests Written (Cumulative)	362+ (target) / [PLACEHOLDER: actual]
Security Tests	[PLACEHOLDER: #]
Code Coverage	87%+ (target) / [PLACEHOLDER: actual %]
Audit Log Entries Types	[PLACEHOLDER: #]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:
- `src/security/symlink-detector.ts`
- `src/security/audit-logger.ts`
- `src/security/file-locker.ts`
- `src/security/race-condition-guard.ts`
- Enhanced policy files
- `tests/security/symlink.test.ts`
- `tests/security/audit-logger.test.ts`
- `tests/security/file-locker.test.ts`
- `tests/security/security.integration.test.ts`
- `docs/security/threat-model.md`
- `docs/security/security-best-practices.md`

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Security Vulnerabilities Found

[PLACEHOLDER: User will list any vulnerabilities discovered during this phase]

Security Mitigations Applied

[PLACEHOLDER: User will list specific mitigations implemented]

Threat Model Updates

[PLACEHOLDER: User will note changes to threat model]

Next Prompt

➡️ **PROMPT 06: Tests + Coverage Enhancement**

PROMPT 06: Tests + Coverage Enhancement

Phase: 1 of 4
Prompt: 06 of 07
Critical Level: 🟡 MEDIUM
Duration Estimate: 2-3 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 05: Security Hardening
- **Next:** PROMPT 07: Documentation + Git + Phase 1 Completion
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation

6. PROMPT 05: Security Hardening
7. ➡️ PROMPT 06: Tests + Coverage Enhancement
8. PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] PROMPT 02 completed
- [ ] PROMPT 02B completed
- [ ] PROMPT 03 completed
- [ ] PROMPT 04 completed
- [ ] PROMPT 05 completed
- [ ] All core functionality implemented
- [ ] Security hardening complete
- [ ] Existing test suite running

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Integration test expansion
- Edge case coverage
- Error scenario testing
- Performance testing
- Concurrency testing
- Boundary condition testing
- Code coverage gap analysis
- Test documentation

Expected Test Categories:
1. **Integration Tests:**
- End-to-end workflows
- Multi-component interactions
- Policy enforcement chains
- Real filesystem operations

1. **Edge Case Tests:**
   - Boundary values
   - Unusual input combinations
   - Resource exhaustion scenarios
   - Timeout conditions

2. **Error Scenario Tests:**
   - All exception paths
   - Recovery mechanisms

   - Cascading failures
   - Partial failure handling

3. **Performance Tests:**
   - Large file operations
   - Bulk operations
   - Memory usage
   - Operation timing

4. **Concurrency Tests:**
   - Parallel operations
   - Lock contention
   - Race conditions
   - Deadlock scenarios

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] Comprehensive integration test suite
- [ ] Edge case tests for all components
- [ ] Error scenario coverage complete
- [ ] Performance test suite
- [ ] Concurrency test suite
- [ ] 472+ total tests
- [ ] 90%+ code coverage
- [ ] Coverage report generated
- [ ] Test documentation
- [ ] Testing best practices guide

Success Criteria

- [ ] Total test count ≥ 472
- [ ] Code coverage ≥ 90%
- [ ] All components have integration tests
- [ ] All edge cases covered
- [ ] All error paths tested
- [ ] Performance benchmarks established
- [ ] Concurrency tests pass reliably
- [ ] No flaky tests
- [ ] Tests run in reasonable time (< 5 minutes)
- [ ] Coverage gaps documented and justified

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Total Tests	472+ (target) / [PLACEHOLDER: actual]
Integration Tests	[PLACEHOLDER: #]
Edge Case Tests	[PLACEHOLDER: #]
Error Scenario Tests	[PLACEHOLDER: #]
Performance Tests	[PLACEHOLDER: #]
Concurrency Tests	[PLACEHOLDER: #]
Code Coverage	90.05%+ (target) / [PLACEHOLDER: actual %]
Test Execution Time	[PLACEHOLDER: X minutes X seconds]
Coverage Improvement	[PLACEHOLDER: +X%]

Coverage Breakdown

[PLACEHOLDER: User will fill in coverage by component]

Expected Breakdown:
Component	Coverage
Exceptions	[PLACEHOLDER: X%]
Policies	[PLACEHOLDER: X%]
PathValidator	[PLACEHOLDER: X%]
FileOperations	[PLACEHOLDER: X%]
DirectoryOperations	[PLACEHOLDER: X%]

MemoryScaffold	[PLACEHOLDER: X%]
Security Components	[PLACEHOLDER: X%]
Overall	90.05%+

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:
- `tests/integration/*.integration.test.ts` (multiple files)
- `tests/edge-cases/*.test.ts`
- `tests/error-scenarios/*.test.ts`
- `tests/performance/*.test.ts`
- `tests/concurrency/*.test.ts`
- `coverage/` directory with reports
- `docs/testing/test-strategy.md`
- `docs/testing/coverage-analysis.md`

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Coverage Gaps Identified

[PLACEHOLDER: User will list any remaining coverage gaps and justification]

Performance Benchmarks

[PLACEHOLDER: User will document performance test results]

Expected Benchmarks:
Operation	Time	Throughput
Read Small File (1KB)	[PLACEHOLDER]	[PLACEHOLDER]
Read Large File (10MB)	[PLACEHOLDER]	[PLACEHOLDER]
Write Small File	[PLACEHOLDER]	[PLACEHOLDER]
Write Large File	[PLACEHOLDER]	[PLACEHOLDER]
Directory Listing (100 items)	[PLACEHOLDER]	[PLACEHOLDER]
Bulk Operations (1000 files)	[PLACEHOLDER]	[PLACEHOLDER]

Next Prompt

➡️ **PROMPT 07: Documentation + Git + Phase 1 Completion**

PROMPT 07: Documentation + Git + Phase 1 Completion

Phase: 1 of 4
Prompt: 07 of 07
Critical Level: 🟢 LOW
Duration Estimate: 1-2 hours
Status: ✅ COMPLETED

🧭 Navigation

- **Previous:** PROMPT 06: Tests + Coverage Enhancement
- **Next:** Phase 2 (Future)
- **Phase Overview:** Phase 1 Overview

📚 Phase 1 Context

Phase 1 Goal: Establish foundational memory scaffold with comprehensive security, guardrails, and verification systems.

All Phase 1 Prompts:
1. PROMPT 01: Development Environment Setup
2. PROMPT 02: TypeScript Project Initialization
3. PROMPT 02B: Quality Gates Setup
4. PROMPT 03: Exception Hierarchy + Guardrails Foundation
5. PROMPT 04: Memory Scaffold Implementation
6. PROMPT 05: Security Hardening
7. PROMPT 06: Tests + Coverage Enhancement
8. ➡️ PROMPT 07: Documentation + Git + Phase 1 Completion

✅ Prerequisites

- [ ] PROMPT 01 completed
- [ ] PROMPT 02 completed
- [ ] PROMPT 02B completed
- [ ] PROMPT 03 completed
- [ ] PROMPT 04 completed
- [ ] PROMPT 05 completed
- [ ] PROMPT 06 completed
- [ ] All tests passing
- [ ] Code coverage ≥ 90%
- [ ] All functionality working

📝 Task Description

Context

[PLACEHOLDER: User will fill in the context/background for why this prompt was needed]

Task

[PLACEHOLDER: User will fill in the specific task details]

Expected Focus Areas:
- Comprehensive README.md
- API documentation
- Architecture documentation
- Usage examples and guides
- Security documentation
- Testing documentation
- Contributing guidelines
- Git repository initialization
- Initial commit with full history
- Phase 1 completion verification

Expected Documentation Files:
1. **README.md** - Project overview, quick start, features
2. **docs/api/README.md** - API reference for all components
3. **docs/architecture/README.md** - System architecture and design decisions
4. **docs/security/README.md** - Security model, threat model, best practices
5. **docs/testing/README.md** - Testing strategy, coverage, running tests
6. **docs/guides/getting-started.md** - Beginner's guide
7. **docs/guides/usage-examples.md** - Common usage patterns
8. **CONTRIBUTING.md** - How to contribute to the project

Deliverables

[PLACEHOLDER: User will list specific deliverables]

Expected Deliverables:
- [ ] README.md with comprehensive project overview
- [ ] Complete API documentation
- [ ] Architecture documentation with diagrams
- [ ] Security documentation
- [ ] Testing documentation
- [ ] Usage guides and examples
- [ ] Contributing guidelines
- [ ] Git repository initialized
- [ ] Initial commit created
- [ ] Phase 1 completion report

Success Criteria

- [ ] README.md covers all essential information
- [ ] API documentation is complete and accurate
- [ ] Architecture is well-documented with diagrams
- [ ] Security model is clearly explained
- [ ] Testing approach is documented
- [ ] Examples demonstrate all major features
- [ ] Git history is clean and meaningful
- [ ] All Phase 1 requirements met

- [ ] Phase 1 metrics achieved (472 tests, 90%+ coverage)
- [ ] Ready for Phase 2

📊 Execution Log

Field	Value
Date Executed	[PLACEHOLDER: YYYY-MM-DD]
Model Used	[PLACEHOLDER: Claude Code / Verdent / etc.]
Start Time	[PLACEHOLDER: HH:MM]
End Time	[PLACEHOLDER: HH:MM]
Actual Duration	[PLACEHOLDER: X hours X minutes]
Status	[PLACEHOLDER: ✅ SUCCESS / ⚠️ PARTIAL / ❌ FAILED]

Metrics

Metric	Value
Documentation Files Created	8+ (target) / [PLACEHOLDER: actual]
Total Documentation Pages	[PLACEHOLDER: #]
Code Examples	[PLACEHOLDER: #]
Diagrams Created	[PLACEHOLDER: #]
Git Commits	[PLACEHOLDER: #]
Files in Initial Commit	[PLACEHOLDER: #]

Output Files

[PLACEHOLDER: User will list files created/modified]

Expected Files:

- `README.md`
- `CONTRIBUTING.md`
- `docs/api/README.md`
- `docs/architecture/README.md`
- `docs/architecture/diagrams/*.png`
- `docs/security/README.md`
- `docs/testing/README.md`
- `docs/guides/getting-started.md`
- `docs/guides/usage-examples.md`

- `.gitignore`
- `.git/` directory (Git initialized)

Issues Encountered

[PLACEHOLDER: User will document any issues, blockers, or challenges]

Resolution Notes

[PLACEHOLDER: User will document how issues were resolved]

Key Learnings

[PLACEHOLDER: User will note important insights or decisions]

Documentation Highlights

[PLACEHOLDER: User will note particularly important documentation sections]

Git Commit History

[PLACEHOLDER: User will list key commits]

Expected Commits:
- Initial project structure
- Exception hierarchy implementation
- Memory scaffold implementation
- Security hardening
- Test suite completion
- Documentation completion

📋 Phase 1 Completion Checklist

Core Functionality

- [ ] 14 exception classes implemented
- [ ] 7 security policies implemented
- [ ] 8 core interfaces defined
- [ ] PathValidator fully functional
- [ ] FileOperations fully functional
- [ ] DirectoryOperations fully functional
- [ ] MemoryScaffold main class operational

Security

- [ ] Symlink detection working
- [ ] Audit logging operational
- [ ] File locking implemented
- [ ] Path validation robust
- [ ] Injection prevention active
- [ ] All security policies enforced

Testing

- [ ] 472+ tests passing
- [ ] 90%+ code coverage achieved

- [ ] Integration tests complete
- [ ] Performance tests complete
- [ ] Security tests complete
- [ ] No flaky tests

Quality

- [ ] TypeScript strict mode enabled
- [ ] ESLint passing with no errors
- [ ] Prettier formatting consistent
- [ ] Quality gates (Husky) active
- [ ] No TypeScript errors
- [ ] No linting warnings

Documentation

- [ ] README.md complete
- [ ] API documentation complete
- [ ] Architecture documented
- [ ] Security model documented
- [ ] Testing documented
- [ ] Usage examples provided
- [ ] Contributing guidelines available

Git

- [ ] Repository initialized
- [ ] Initial commit created
- [ ] .gitignore configured
- [ ] Commit history clean
- [ ] All files tracked

🎯 Phase 1 Final Metrics

Metric	Target	Actual
Exception Classes	14	[PLACEHOLDER]
Security Policies	7	[PLACEHOLDER]
Core Interfaces	8	[PLACEHOLDER]
Total Tests	472+	[PLACEHOLDER]
Code Coverage	90%+	[PLACEHOLDER]
Documentation Files	8+	[PLACEHOLDER]
Source Files	-	[PLACEHOLDER]
Test Files	-	[PLACEHOLDER]

🚀 Readiness for Phase 2

[PLACEHOLDER: User will assess readiness for Phase 2]

Phase 2 Preview:
Phase 2 will focus on [PLACEHOLDER: brief description of Phase 2 goals]

Blockers/Concerns:
[PLACEHOLDER: Any concerns or blockers before starting Phase 2]

Next Steps

1. [PLACEHOLDER: Next action items]
2. [PLACEHOLDER: Any cleanup needed]
3. [PLACEHOLDER: Phase 2 preparation]

📚 Appendix

Template Usage Guide

This master document serves as:
1. **Template Reference:** Use sections as templates for individual prompt documents
2. **Completion Tracking:** Fill in placeholders as you document each prompt
3. **Historical Record:** Maintain accurate record of Phase 1 completion
4. **Planning Tool:** Review structure before starting Phase 2

How to Use This Template

For Individual Prompt Documentation:

1. Copy the relevant prompt section
2. Create file: `docs/phases/phase_1/prompt_XX/README.md`
3. Fill in all `[PLACEHOLDER]` sections with actual data
4. Add specific details, code snippets, screenshots as needed
5. Link to actual files created (use relative paths)

For Retroactive Documentation:

1. Review git history for commit dates and times
2. Check test reports for coverage and test count data
3. List all files created/modified from git log
4. Document issues from memory or issue tracker
5. Capture key decisions from code review notes
6. Fill in all metrics from final state of code

For Phase Completion:

1. Verify all checkboxes in completion checklist
2. Fill in final metrics table
3. Document any deviations from plan
4. Note lessons learned
5. Prepare Phase 2 planning based on Phase 1 experience

Document Structure Best Practices

File Organization

```
docs/
├── phases/
│   ├── phase_1/
│   │   ├── README.md (Phase 1 Overview)
│   │   ├── prompt_01/
│   │   │   └── README.md (PROMPT 01 documentation)
│   │   ├── prompt_02/
│   │   │   └── README.md (PROMPT 02 documentation)
│   │   ├── prompt_02b/
│   │   │   └── README.md (PROMPT 02B documentation)
│   │   ├── prompt_03/
│   │   │   └── README.md (PROMPT 03 documentation)
│   │   ├── prompt_04/
│   │   │   └── README.md (PROMPT 04 documentation)
│   │   ├── prompt_05/
│   │   │   └── README.md (PROMPT 05 documentation)
│   │   ├── prompt_06/
│   │   │   └── README.md (PROMPT 06 documentation)
│   │   └── prompt_07/
│   │       └── README.md (PROMPT 07 documentation)
│   └── phase_2/
│       └── ... (Future)
```

Linking Between Documents

Use relative links for navigation:

```
[Next Prompt](../prompt_02/README.md)
[Phase Overview](../README.md)
[Previous Prompt](../prompt_01/README.md)
```

Code Examples

Include actual code snippets where relevant:

```
// Example from implementation
const result = await memoryScaffold.readFile('/path/to/file.txt');
```

Diagrams and Visuals

- Add architecture diagrams to `docs/architecture/diagrams/`
- Reference with relative paths: `![Architecture](/architecture/diagrams/overview.png)`
- Use mermaid diagrams in markdown when possible

Metrics Collection Tips

Test Metrics

```
# Get test count
pnpm test --reporter=verbose | grep -c "✓"

# Get coverage
pnpm test:coverage
# Check coverage/coverage-summary.json
```

File Counts

```
# Count source files
find src -name "*.ts" | wc -l

# Count test files
find tests -name "*.test.ts" | wc -l

# List all files created
git ls-files
```

Git History

```
# Get commit dates
git log --pretty=format:"%h - %an, %ar : %s"

# Get files changed in commit
git show --name-only <commit-hash>

# Get commit count
git rev-list --count HEAD
```

Quality Checklist

Before considering documentation complete:
- [ ] All placeholders filled in
- [ ] Actual metrics from final codebase
- [ ] Real file paths (not examples)
- [ ] Accurate dates and times
- [ ] Issues documented with resolutions
- [ ] Key learnings captured
- [ ] Links working (relative paths correct)
- [ ] Code examples tested
- [ ] Diagrams included where needed
- [ ] Spelling and grammar checked

Document Version: 1.0
Created: [Date this master template was created]
Last Updated: [Date of last modification]
Status: Template (Ready for retroactive documentation)

🎓 Phase 1 Lessons Learned

[PLACEHOLDER: After completing all prompt documentation, add high-level lessons learned across entire Phase 1]

What Went Well

[PLACEHOLDER: Key successes]

What Could Be Improved

[PLACEHOLDER: Areas for improvement]

Recommendations for Future Phases

[PLACEHOLDER: Recommendations based on Phase 1 experience]

Time Management Insights

[PLACEHOLDER: Actual time vs estimated time analysis]

Technical Debt Identified

[PLACEHOLDER: Any technical debt that should be addressed]

End of Phase 1 Master Document Template